

Laboratorio II

Corso A

Lezione 6

- Continuo prima esercitazione
- Programmazione C di base
 - Funzioni e funzioni di I/O base
 - Puntatori
 - Passaggio per valore e per riferimento
 - Array e puntatori

Esercizi

- Scrivere un programma che legge due vettori di dimensione ***n*** fissa, ad esempio 10, e fa il prodotto scalare tra i due vettori. Il prodotto scalare è la somma dei prodotti di ogni coppia di elementi dei due vettori dello stesso indice, cioè $a[0]*b[0] + \dots + a[n]*b[n]$
- Scrivere un programma che legge ***n*** e due matrici quadrate di dimensione ***nXn***, ad esempio di 10 elementi, e fa il prodotto scalare tra le due matrici.
- Scrivere un programma che fa la somma dei parametri ricevuti dalla linea di comando

Esercizi

- Scrivere un programma che inverta gli elementi di un array di interi A, di dimensione 10. Il programma deve invertire l'array senza utilizzare un array di appoggio, ossia scambiare il contenuto della prima e dell'ultima cella, della seconda e della penultima, ecc... ed infine stampare a video ogni singolo elemento dell'array modificato.
- Si scriva un programma di gestione dell'anagrafe dei dipendenti di un'azienda che legga i seguenti dati per 6 diversi dipendenti:
 - Un codice identificativo del dipendente (intero), **diverso** per ogni dipendente
 - L'età del dipendente in anni (un intero)
 - Stipendio (un float, indicato in euro come segue: 16540.32);
 - Il tipo di dipendente, indicando con il numero 0 i dirigenti, 1 i quadri, 2 gli operai e 3 gli impiegati, usando una opportuna `enum` con etichette rispettivamente **dirigente**, **quadro**, **operaio** e **impiegato**.

Per ogni dipendente, si chiedi in input il codice identificativo, controllando se è diverso da uno già immesso e richiedendolo se è diverso, l'età, lo stipendio e il tipo di dipendente.

Nello stesso ordine in cui sono stati immessi i dati dei dipendenti si stampino, uno per linea, il codice identificativo, l'età e il ruolo di ogni dipendente (l'etichetta) che guadagni meno dello stipendio medio aritmetico di tutti i dipendenti dell'azienda. I valori stampati devono essere separati da un unico spazio.

Esempio di Input:

35,1,56,55650.33, 0, 2, 28, 36540.56, 1, 3, 22, 22120.54, 2, 4, 32, 28450.45, 3, 5, 45, 18960.65, 2, 6, 33,38960.54, 1

Esempio di Output:

3 22 operaio

4 32 impiegato

Esercizi: soluzione 1

```
#include <stdio.h>
int main()
{
    int i,j,scambio;
    int A[10];

    for ( i = 0; i < 10; i++ )
        A[i]=i;

    /* Inversione, scorro l'array fino alla metà della sua dimensione */
    for ( i = 0; i < 10/2; i++ )
    {
        j = (10-1)-i;
        scambio = A[i];
        A[i] = A[j];
        A[j] = scambio;
    }

    /* Output */
    for ( i = 0; i < 10; i++ )
        printf("%d\n", A[i]);
    return 0;
}
```

Esercizi: soluzione 2

```
#include <stdio.h>
#include <stdlib.h>

typedef enum
{
    dirigente,
    quadro,
    operaio,
    impiegato
} ruolo_t;

typedef struct
{
    int id;
    int eta;
    float salario;
    ruolo_t ruolo;
} dipendente_t;

int main(int argc, char *argv[])
{
    int i, id_dip, eta_dip, ruolo_dip, eta;
    float media = 0, salario_dip;

    dipendente_t dipendenti[6];
    scanf("%d",&eta);
```

```
    for (i=0; i<6; i++) {
        do {
            scanf("%d",&id_dip);
            dipendenti[i].id = id_dip;
            int trovato = 0;
            for (int j=0;j<i;j++) {
                if(id_dip == dipendenti[j].id) {
                    printf("Id %d del dipendente esistente, reinserirlo!\n",id_dip);
                    trovato = 1;
                    break;
                }
            }
            if(!trovato) {
                scanf("%d",&eta_dip);
                scanf("%f",&salario_dip);
                scanf("%d",&ruolo_dip);
                dipendenti[i].eta = eta_dip;
                dipendenti[i].salario = salario_dip;
                dipendenti[i].ruolo = ruolo_dip;
            }
            while(!trovato);
            media += salario_dip;
        }
        media /= 6;
        for (i=0; i<6; i++)
            if (dipendenti[i].salario < media && dipendenti[i].eta <= eta){
                printf("%d ", dipendenti[i].id);
                printf("%d ", dipendenti[i].eta);
                switch(dipendenti[i].ruolo){
                    case dirigente:
                        printf("dirigente\n");
                        break;
                    case quadro:
                        printf("quadro\n");
                        break;
                    case operaio:
                        printf("operaio\n");
                        break;
                    default:
                        printf("impiegato\n");
                }
            }
    }

    return 0;
}
```

Funzioni

- Astrazioni di blocchi di codice con un nome
 - Implementano una certa funzionalità, per essere riutilizzata
- Utili per realizzare la metodologia di programmazione “top-down”
- Un programma è costituito da uno o più files contenenti zero o più funzioni, di cui la prima ad essere eseguita è la funzione `main`, che deve sempre essere presente
- **Definizione** - include il corpo della funzione con il codice da eseguire.

```
tipo nome_funzione ( lista_parametri ) {  
      
}
```

 - `tipo`
 - tipo del valore di ritorno. Non è possibile più di un valore
 - Si può usare la parola chiave `void` per indicare che non c'è un valore di ritorno
 - `nome_funzione`
 - Stesse regole degli identificatori
 - `lista_parametri`
 - Elenco di coppie `tipo identificatore`
 - Ogni coppia è separata da un operatore virgola ,
 - Sono chiamati **parametri formali** e possono essere usati localmente nella funzione
 - Se non c'è una lista di parametri, si può usare la parola chiave `void` o non indicare nessun parametro:
`int main()`
- Ogni funzione è identificata univocamente dai 3 elementi suddetti

```
#include <stdio.h>  
  
int max(int, int);  
  
int main(void) {  
      
    int a = 5, b=10;  
      
    printf("%d\n", max(a,b));  
}  
  
int max(int a, int b){  
    if (a>b)  
        return a;  
    return b;  
}
```

Funzioni

- **Dichiarazione** - tipicamente all'inizio del programma - parametri e tipo di ritorno - prototipo
 - **Prototipo**
 - Informa il compilatore sul numero di parametri, del loro tipo e del tipo di ritorno di una funzione definita in questo o in un altro file (incluso)
- Posso dichiarare e definire allo stesso tempo prima della main - di solito si separano (anche in file diversi e.g. .h, .c)
- Se manca la definizione => linker error
- Se manca la dichiarazione => compiler error
- `return`
 - Termina l'esecuzione della funzione tornando all'ambiente del chiamante
 - Si ritorna sempre all'istruzione successiva a quella in cui la funzione viene invocata
 - `return;`
 - Usata quando non c'è un valore di ritorno
 - `return espressione;`
 - Usata quando c'è un valore di ritorno
 - Il valore calcolato da espressione deve essere dello stesso tipo indicato dalla funzione
 - Viene fatto un cast implicito se necessario, e se possibile altrimenti **errore**
- In mancanza di una istruzione `return`, la funzione termina alla fine del suo blocco più esterno. Non viene ritornato un valore

```
#include <stdio.h>

int max(int, int);

int main(void) {

    int a = 5, b=10;

    printf("%d\n", max(a,b));

}

int max(int a, int b){
    if (a>b)
        return a;
    return b;
}
```


Funzioni: invocazione di funzione

nome_funzione(lista_parametri)

- Una funzione viene invocata scrivendo il suo nome seguito dalla lista dei **parametri attuali**
 - I parametri attuali sono una lista di espressioni separate da una virgola
 - **Devono essere nello stesso ordine e valutare allo stesso tipo del parametro formale corrispondente della funzione invocate**
 - Effettuato casting implicito dove possibile
 - Sono valutati nell'ambiente del chiamante
- Costituisce una espressione: il suo valore è il valore di ritorno della funzione
`max(somma( b, c ), a)`
- Al termine della funzione, il programma riprende dall'istruzione successiva al punto della sua invocazione
- In C Il passaggio di parametri viene fatto sempre per **valore**
 - si passa una copia del valore della variabile

```
#include <stdio.h>

int max(int, int);

int main(void) {

    int a = 5, b=10;

    printf("%d\n", max(a,b));

}

int max(int a, int b){
    if (a>b)
        return a;
    return b;
}
```

Qualche funzione per I/O

- Libreria standard per I/O: `stdio`
- Per mostrare a video (stdout) dei valori - funzione `printf()`
 - `printf(stringa_formattazione)`
 - scrive la stringa passata come argomento.
 - la stringa di formattazione può contenere dei segnaposti (`% . . .`) per includere dei valori di variabili - le variabili vanno incluse dopo la stringa, in ordine

```
#include <stdio.h>

#define LEN 20

struct Studente {
    char nome[LEN];
    int eta;
};

int main(void) {

    struct Studente s = { "Antonio",15};

    printf("Nome: %s, Età: %d\n",s.nome, s.eta);
    printf("Dimensione in memoria: %lu\n", sizeof(s));

    return 0;
}
```

Printf: segnaposti

- il segnaposto dipende dal tipo della variabile da scrivere
 - interi: `%d`, `%u` (unsigned). Si antepone `h` per `short` e `l` per `long`
 - reali: `%f`, `%e` (notazione scientifica), `%g` la piu' breve tra notazione standard e scientifica. Per i `long double` si antepone `L`
 - caratteri: `%c`
 - stringhe: `%s`
 - si possono includere opzioni di formattazione
 - ampiezza minima del campo `"%5d"`
 - numero di decimali dopo la virgola `"%.5f"`

Qualche funzione per I/O

- Per leggere da stdin - funzione `scanf()`
- Sintassi simile alla `printf`, prende una stringa di formattazione più gli **indirizzi (&)** delle variabili dove salvare i valori letti.
 - `scanf("%d %d", &a, &b)` - legge due interi e li salva nelle variabili `a` e `b`
- I segnaposti sono gli stessi di `printf` tranne quello per double: `"%lf"`

```
#include <stdio.h>

int main(void) {
    int a=0;
    double b=0;
    scanf("%d %lf",&a,&b);
    printf("a=%d\nb=%f\n", a,b);
    return 0;
}
```

```
> ./main
7
9.8
a=7
b=9.800000
> 
```

Domande?

